

stored inside the MBR, and the next boot-strap procedures are performed by it. At this point, the BIOS is only responsible for reading and writing sectors specified by the operating system, i.e., by calling service functions of INT 13h interrupt.

The UEFI BIOS can work on a file system level. Therefore, in order to load the operating system, it's enough to read and execute a file (by default, `EFI\BOOT\bootx64.efi` for x64 UEFI). Support for the file system at the firmware level is important if we decide to integrate into the BIOS body such functions that require reading or writing of files—for example, in-house reflashing of the BIOS requires us to read the BIOS image file directly from the disk. Besides, if the BIOS offers ready-to-use file operations instead of disk sector operations, it's much easier to implement more complex functions, like a network boot.

The third advantage: A library with various ready-to-use functions

When it comes to programming diagnostic software, you have to deal with systems resources that have different implementations within multiple hardware platforms. They are usually built with variable sets of system logic. For instance, to read those parameters from RAM modules, you need to read the contents of the Serial Presence Detect chip, known as SPD EPROM (this is a ROM with 256 bytes located on the memory module). In order to read SPD, it is necessary to work with the System Management Bus (SMBus) controller, which in turn, is part of the systems logic. For each platform, this controller is implemented in different ways, so a DOS program would necessarily have a mix of drivers for each group of platforms. The legacy BIOS does not provide service procedures to perform such read/write operations.

In turn, the UEFI BIOS does provide such service routines (in this example, SMBus Host Controller Protocol), so a UEFI-application can simply call an available procedure that is part of the firmware, and it shouldn't work directly with the controller's registers. It is not necessary to write your own driver for each platform any more. SMBus operations are just one small example. Some of the prominent examples are: access routines to USB/LAN controllers, access control to BIOS logic chips, etc. For these and many other devices, the UEFI BIOS provides a huge amount of ready-to-use service procedures.

The fourth advantage: Versatility

Legacy BIOS service routines use a parameters-passing model that is tightly tied up to a set of x86 processor registers. However, UEFI-protocols are based on a universal hardware-abstracted parameters-passing model. That's why UEFI implementation is available for non-x86 processor architecture as well, such as Itanium and ARM.

Let's write a sample application

Now, let's analyse two examples of how to output text strings. One application (HELLO32.ASM) is for the IA32

EFI environment, and the other (HELLO64.ASM, a full source code can be downloaded from http://www.linuxforu.com/article/source_code/jan14/uefi.zip) is for x64 UEFI.

Here's a brief description of the program's actions.
Step 1: When a parent task (whether it is the UEFI Shell or the firmware itself) loads our application, it passes a pointer to the *EFI System Table*. How should we read this pointer? Easy! For IA32 EFI, this is a 32-bit word in stack at [ESP+8] offset and for x64 UEFI, accordingly, a 64-bit word in RDX register.

```
format pe dll
entry main

section '.text' code executable readable
main:
;--- EFI Handoff state at application entry point ----;
; dword ss:[ESP+08] = Return address for RET NEAR, 32-bit ;
; dword ss:[ESP+04] = Pointer to EFI_HANDLE for this application ;
; dword ss:[ESP+08] = Pointer to EFI_SYSTEM_TABLE ;
;-----;
;--- Prepare parameters for string write ---
        mov     ecx,[esp+08h] ; ECX = EFI_SYSTEM_TABLE address
        mov     ecx,[ecx+2Ch] ; ECX = EFI_SYSTEM_TABLE.ConOut
        mov     edx,_Hello ; EDX = Message address
        ush     edx,ecx ; Push input parameters
;----- Parameters for EFI Service function call -----;
; dword ss:[ESP+08] = Return address (pushed by CALL) ;
; dword ss:[ESP+04] = Function offset (push ecx) ;
; dword ss:[ESP+08] = String address (push edx) ;
;-----;
;--- Call function ---
        call    dword [ecx+4] ; SIMPLE_TEXT_OUTPUT_
INTERFACE.OutputString
        add     esp,8 ; Remove parameters
;--- Exit ---
        xor     eax,eax ; EAX=EFI_STATUS
        retn

section '.data' data readable writable
_Hello du 'Debug sample for EFI_SIMPLE_TEXT_OUTPUT_
PROTOCOL.',13,10
        du 'IA32 EFI Edition.',13,10
        du 'EFI application written in FASM.',13,10

section '.reloc' data discardable fixups
```

Step 2: From the EFI System Table our program should retrieve a pointer to the interface unit of *EFI Simple Text Output Protocol* (see Figure 1). In our examples, this pointer is named `EFI_SYSTEM_TABLE.ConOut` or `Console Output`.

How can we calculate offsets in *EFI System Table*? We must add two numbers A and B, where: A is a header size for *EFI System Table*, that is equal to 24 bytes for both implementations, IA32 and x64; and B is the `ConOut` pointer's offset, that is sized differently